UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Data representation and number systems

Answer: 286610 = 54628

- *1 mark for identifying the target base.*
- *1 mark for a correct repeated-division or place-value method.*
- *1 mark for the correct digit sequence.*
- *1 mark for the correct base subscript.*
- *1 mark for a complete, consistent workflow.*

QUESTION A2 (5 marks)

Topic: Computer generations and history

Source: CSC429 (17).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The successor of the Difference Engine was the Analytical Engine, also designed by Charles Babbage. It was a general-purpose programmable mechanical computer, whereas the Difference Engine was a special-purpose calculator.

(b) The Difference Engine was a special-purpose mechanical calculator designed to automatically compute and print mathematical tables (e.g., for naval navigation) using the method of finite differences; it performed only addition and subtraction of polynomial terms and had no general programmability. The Mechanical Calculating Machine (as the term is used in the history of computing, often referring to later mechanical calculators such as the arithmometer or Babbage's earlier designs) was a more general-purpose mechanical device for performing the four basic arithmetic operations (addition, subtraction, multiplication, division) on numbers entered by an operator; it did not automatically compute tables but was a general calculating tool. In short, the Difference Engine was a dedicated table-producing machine, while the Mechanical Calculating Machine was a general-purpose arithmetic calculator.

(c) The Difference Engine belongs to the pre-electronic mechanical era (often called the "mechanical" or "first generation" in a broad historical sense, though the term "first generation" usually refers to vacuum-tube computers). The available technology was mechanical: gears, levers, cams, and rods driven by a hand crank. There were no electronic components; power was human or steam, and data were represented by the positions of mechanical parts. Manufacturing precision was a limiting factor, and Babbage's designs were never fully completed in his lifetime.

Unofficial marking guide:

- *1 mark: (a) Correctly identifies the Analytical Engine as the successor.*
- *1 mark: (b) States that the Difference Engine was special-purpose (automatic table computation using finite differences).*
- *1 mark: (b) States that the Mechanical Calculating Machine was general-purpose (four arithmetic operations) or contrasts programmability/function.*
- *1 mark: (c) Identifies the technology as mechanical (gears, levers, cams, hand crank).*
- *1 mark: (c) Explains that there were no electronic components (pre-electronic era) or notes the limitations of mechanical technology.*

Model confidence: medium.

Model note: The term 'Mechanical Calculating Machine' is not a standard proper name; it is interpreted as a general mechanical calculator (e.g., arithmometer or Babbage's earlier designs) to contrast with the special-purpose Difference Engine. The 'computer generation' of the Difference Engine is taken as the mechanical era (pre-electronic), not the conventional first generation (vacuum tubes).

QUESTION A3 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (13).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three characteristics of a Symmetric Multiprocessor (SMP):

1. Two or more identical processors sharing a single main memory. All CPUs are of the same type and have equal access to a common, globally shared physical memory (typically via a shared bus or

© Hak Cipta Universiti Teknologi MARA

interconnection network), so any processor can access any memory location.

2. Symmetric access and equal status — no master/slave relationship. Every processor runs the same operating-system kernel and can execute any task, including kernel/OS code; all processors are peers and share the same I/O devices and system resources.

3. Tightly coupled, single-system image with hardware-supported coherence. The processors cooperate through shared memory under one OS instance, presenting a single system image to users; cache coherence and synchronisation (e.g. locking) are maintained in hardware/OS so shared data stays consistent.

(Other acceptable characteristics: shared single OS, common bus/interconnection, load balancing of processes across CPUs, scalability by adding processors.)

(b) Advantage of SMP in terms of availability:

Because all processors are peers with no single master, the system can continue operating (gracefully degrade) if one processor fails. The remaining processors keep running the OS and the workload, so the failure of a single CPU does not bring the whole system down — the system remains available, merely with reduced processing capacity. This redundancy of processing resources gives higher availability/fault tolerance than a single-processor system, where a CPU failure halts the entire machine.

Unofficial marking guide:

• 1 mark: states that SMP has two or more identical/symmetric processors sharing a single main memory
• 1 mark: states that all processors have equal status/access with no master-slave relationship (same OS, any processor can run any task)
• 1 mark: states tight coupling / single-system image with shared resources and hardware-supported coherence or synchronisation
• 1 mark: identifies that failure of one processor does not stop the system (no single point of failure / graceful degradation)
• 1 mark: explains that remaining processors continue running the OS/workload, so the system stays available with reduced capacity
Model confidence: high.
Model note: Unofficial model answer, not an official marking scheme. The question text has a minor OCR/spacing error ('interms'); interpreted as 'in terms of'. Any three valid SMP characteristics earn the 3 marks in part (a).

QUESTION A4 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A holds the actual/effective address of the operand, so the operand is fetched directly from memory location A — the address field contains the address itself, not a pointer to it.

(b) How it is used: The CPU decodes the instruction and takes the address field A as the effective address (EA = A). It then issues a memory read to location A, and the contents of that location (the operand) are loaded into the CPU (e.g. into a register or the accumulator) for use by the instruction. For a store, the result is written back to location A. No further address calculation or extra memory access for address resolution is needed — one memory reference fetches the operand. Example: LOAD 500 fetches the operand stored at memory address 500.

(c) Advantage: It is simple and fast — no extra address computation and only one memory access to obtain the operand (the address is available immediately from the instruction). Disadvantage: The address field must be large enough to hold a full memory address, limiting the range of addresses (and instruction size), and it offers limited flexibility — it cannot easily support relocation, arrays/pointers or a large address space without long instructions.

Unofficial marking guide:

• 1 mark: Correctly names the technique as direct (absolute) addressing.
• 1 mark: States EA = A, i.e. the address field holds the actual operand address.
• 1 mark: Explains the fetch/use — CPU reads memory at address A to obtain the operand (one memory reference, no extra address calculation).
• 1 mark: Gives ONE valid advantage (e.g. simple/fast, single memory access, no address computation).
• 1 mark: Gives ONE valid disadvantage (e.g. limited address range/large address field, inflexible, no relocation/pointer support).

© Hak Cipta Universiti Teknologi MARA

Model confidence: high.
Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A, then Memory: Operand) to depict direct addressing; if the figure actually shows an arrow from the address field to a memory location containing a further address, the answer would instead be indirect addressing. Assumed the standard direct-addressing interpretation consistent with the shown layout.

QUESTION A5 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (6).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three main key factors for memory selection [6 marks]

1. Cost per bit: Memory is selected so that the cost per bit falls as one moves down the hierarchy. Fast semiconductor memory (SRAM, registers, cache) has a high cost per bit, while slower, denser memory (DRAM main memory, magnetic disk, optical/tape secondary storage) has a much lower cost per bit. A designer balances total cost against required capacity and speed.

2. Capacity: The required storage size (in bytes/words) is a key selection factor. Larger capacity is needed for main memory and secondary storage, and capacity interacts with addressing (address bus width) and with cost. Insufficient capacity causes excessive swapping/paging and poor performance.

3. Access time / speed (performance): Memory must match the speed of the processor and the level of the hierarchy it serves. Access time, memory cycle time and transfer rate determine whether the memory can keep the CPU supplied; faster memory (cache/SRAM) is chosen close to the CPU, slower but cheaper memory further away. Other acceptable factors include volatility, erasability/reprogrammability, physical size/form factor, power consumption and reliability, but the three above are the main ones.

(b) Examples [4 marks]

Internal cabling memory (memory used with/inside cabling or as removable/internal connection media):
- USB flash drive (pen drive)
- Memory card (SD card / microSD / CompactFlash)
(Also acceptable: external/portable hard disk, external SSD, ROM cartridge.)

System memory (main memory directly addressable by the CPU):
- RAM (DRAM/SDRAM/DDR main memory)
- ROM (BIOS/firmware ROM, PROM/EPROM/EEPROM)
(Also acceptable: cache memory, SIMM/DIMM modules.)

Unofficial marking guide:

- *1 mark: identifies cost per bit as a factor*
- *1 mark: explains cost per bit across the memory hierarchy*
- *1 mark: identifies capacity as a factor*
- *1 mark: explains capacity and its effect (cost/addressing/performance)*
- *1 mark: identifies access time/speed (or another valid factor such as volatility) as a factor*
- *1 mark: explains access time/speed and its role in the hierarchy*
- *1 mark: one correct internal cabling memory example (e.g. USB flash drive)*
- *1 mark: second correct internal cabling memory example (e.g. memory card)*
- *1 mark: one correct system memory example (e.g. RAM)*
- *1 mark: second correct system memory example (e.g. ROM)*

Model confidence: medium.
Model note: The phrase 'internal cabling memory' is unusual/possibly an OCR or translation artefact; it is interpreted as removable/internal connection memory media (USB flash drives, memory cards). If the lecturer intended a different category, the examples should be adjusted. Part (a) accepts alternative valid factors (volatility, reliability, power) provided they are discussed.

QUESTION A6 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (18).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Differentiate CPU involvement in interrupt-driven I/O versus DMA (Direct Memory Access — the question's 'Direct Manipulation Access' is taken to mean DMA). In interrupt-driven I/O the CPU is heavily involved: it issues the I/O command, then either busy-waits or continues other work, but every unit of data transferred requires the device to raise an interrupt, so the CPU must stop, save state, run the

interrupt service routine, transfer/process the data word, and resume. The CPU therefore handles each data item and pays interrupt overhead per transfer, making it inefficient for large blocks. In DMA the CPU involvement is minimal: it sets up the DMA controller (source/destination addresses, transfer count, direction), then the DMA controller transfers the whole block directly between device and memory, stealing memory cycles (cycle stealing) without CPU intervention. The CPU is interrupted only once, at the end of the entire block transfer, to be told completion. Thus interrupt-driven I/O = CPU involved per byte/word with frequent interrupts; DMA = CPU involved only at setup and completion, with bulk transfer done by the DMA controller.

(b) Definition of interrupt: An interrupt is a signal (hardware or software) sent to the processor that temporarily suspends the currently executing program, causes the CPU to save its state and transfer control to a special routine (the interrupt service routine / handler) that services the interrupting event, after which the CPU restores state and resumes the interrupted program. It allows the CPU to respond promptly to asynchronous events (I/O completion, errors, timers) without polling.

(c) What happens during a context switch: The OS saves the complete execution context (CPU registers, program counter, stack pointer, status/flag register, and memory-management information) of the currently running process into its Process Control Block (PCB). It then selects the next process to run (via the scheduler), loads that process's saved context from its PCB back into the CPU registers, and resumes execution at the restored program counter. During the switch the CPU is doing OS overhead work and no useful user-process work is done; the switch is typically triggered by an interrupt, trap, or system call.

(d) Benefit of a context switch: It enables multitasking/multiprogramming — the CPU can be shared among several processes so that when one process blocks (e.g., waiting for I/O) or its time slice expires, another ready process runs, keeping the CPU busy and improving utilisation and throughput. It also provides responsiveness (interactive systems), fair sharing of CPU time, and supports protection/isolation by cleanly separating process states. The cost is overhead (saved/restored state and cache/TLB effects), so the benefit is improved concurrency and CPU utilisation despite that overhead.

Unofficial marking guide:

• 1 mark: interrupt-driven I/O — CPU issues command and is interrupted for each data item/word, servicing each transfer in the ISR (per-transfer CPU involvement).

• 1 mark: interrupt-driven I/O — frequent interrupts and high CPU overhead, inefficient for large blocks.

• 1 mark: DMA — CPU only sets up the DMA controller (addresses, count, direction) and is interrupted once at completion; bulk transfer done by DMA controller directly to/from memory.

• 1 mark: DMA — cycle stealing / minimal CPU involvement, efficient for large block transfers; clear contrast with interrupt-driven I/O.

• 2 marks: interrupt defined as a signal that suspends the current program, saves state, transfers control to an ISR/handler, then resumes (1 mark for signal/suspension, 1 mark for ISR handling and resume).

• 2 marks: context switch — saving current process context (registers, PC, status) to its PCB (1 mark) and restoring/loading the next process's context from its PCB to resume it (1 mark).

• 2 marks: benefit — enables multitasking/multiprogramming and keeps CPU busy when a process blocks or its time slice expires, improving utilisation/throughput/responsiveness (2 marks; 1 mark for a partial statement such as 'allows multiple processes to share the CPU').

Model confidence: high.

Model note: The question writes 'Direct Manipulation Access'; this is conservatively read as Direct Memory Access (DMA), the standard term in this topic. No figure or table is required. This is an unofficial model answer, not an official marking scheme.

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (8).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Complete machine cycle with the exact location of each phase [6 marks]

The machine cycle is the sequence of operations the CPU performs to fetch and execute one instruction. It is conventionally drawn as a cycle of phases located between the CPU and main memory, with the control unit (CU) and ALU inside the CPU and the instruction/data held in memory. A complete labelled diagram should show:

- A CPU box containing the Control Unit (CU), ALU and registers (PC, MAR, MBR/MDR, IR, ACC).
- A Main Memory box connected to the CPU by the address bus, data bus and control bus.
- The cycle drawn as a loop with the phases in order:

1. Instruction Fetch (IF) — located at the start of the cycle, between memory and the CPU: MAR <- PC; address placed on address bus; memory read; MBR <- M[MAR]; IR <- MBR; PC <- PC + 1.
2. Instruction Decode (ID) — located inside the Control Unit: the opcode in IR is decoded to determine the operation and operands.
3. Operand Fetch / Effective Address Calculation — located between the CPU and memory (or registers): any required operand is read from memory or a register into the ALU/registers.
4. Execute (EX) — located inside the ALU: the operation is performed (arithmetic/logic) or the control unit issues control signals for I/O or branch.
5. Memory Access (MEM) — located between CPU and memory, only if the instruction reads/writes memory (load/store).
6. Write Back (WB) — located at the end of the cycle, writing the result back to a register or to memory.

The loop then returns to Instruction Fetch for the next instruction. (A common simpler four-phase drawing is Fetch -> Decode -> Execute -> Store/Write-back, with the fetch phase located at the memory-CPU interface and execute inside the ALU.)

(b) Description of each phase [4 marks]

1. Instruction Fetch (IF): The Program Counter (PC) holds the address of the next instruction. This address is copied to the Memory Address Register (MAR) and sent on the address bus; the control unit issues a memory-read signal; the instruction is returned on the data bus into the Memory Buffer Register (MBR) and then into the Instruction Register (IR). The PC is incremented to point to the next instruction.

2. Instruction Decode (ID): The control unit examines the opcode field of the IR and determines which operation is required, which operands/addressing mode are involved, and generates the corresponding control signals for the rest of the cycle.

3. Operand Fetch / Execute (EX): Any operands needed are fetched from registers or memory (effective address calculation for indexed/indirect addressing). The ALU then performs the specified arithmetic or logic operation, or the control unit performs a branch/IO action.

4. Write Back / Store (WB): The result produced by the ALU is written back into a register (e.g. accumulator) or stored into memory via the MBR and data bus. The cycle then repeats from the fetch phase for the next instruction.

Unofficial marking guide:

• 1 mark: correct overall cycle drawn as a loop with phases in correct order (Fetch, Decode, Execute, Write-back/Store)
• 1 mark: CPU internals correctly located (CU, ALU, registers PC/MAR/MBR/IR) and memory shown separately
• 1 mark: buses (address, data, control) shown connecting CPU and memory
• 1 mark: exact location of Fetch phase shown at memory-CPU interface with PC/MAR/MBR/IR
• 1 mark: exact location of Decode phase shown inside the control unit
• 1 mark: exact location of Execute phase shown inside the ALU and Write-back at the end of the cycle
• 1 mark: accurate description of the Instruction Fetch phase (PC -> MAR -> memory -> MBR -> IR, PC increment)
• 1 mark: accurate description of the Instruction Decode phase (opcode decoded, control signals generated)
• 1 mark: accurate description of the Execute phase (ALU operation / operand fetch / effective address)
• 1 mark: accurate description of the Write-back/Store phase (result to register or memory)

Model confidence: high.

Model note: The question asks for a drawing; the extracted text contains no figure, so the answer describes the diagram that should be drawn. Either a 4-phase (Fetch-Decode-Execute-Store) or a 6-phase (IF-ID-OF-EX-MEM-WB) cycle is acceptable; the marking guide credits the standard 4-phase version. Marks are allocated 6 for part (a) and 4 for part (b) as stated.

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (2).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Registers are small, high-speed storage locations inside the CPU used for temporary data and control information. Five common types are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch (or loaded on a branch/jump). (2) Instruction Register (IR): holds the instruction currently being decoded and executed; its contents are fed to the control unit. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed (read or write); it connects to the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data word being transferred to or from memory; it connects to the data

© Hak Cipta Universiti Teknologi MARA

bus. (5) Accumulator (AC): a general-purpose arithmetic register that holds one operand and the result of arithmetic/logic operations in a simple CPU. Other acceptable registers include the Stack Pointer (SP), Status/Flag register (PSW), and general-purpose registers (GPRs).

Unofficial marking guide:

- *1 mark: Program Counter (PC) – holds address of next instruction; incremented after fetch.*
- *1 mark: Instruction Register (IR) – holds current instruction for decoding/execution.*
- *1 mark: Memory Address Register (MAR) – holds address of memory location to be accessed.*
- *1 mark: Memory Data Register (MDR/MBR) – holds data read from or written to memory.*
- *1 mark: Accumulator (AC) – holds operand/result of arithmetic and logic operations.*
- *5 marks: one mark for each correctly named register with a correct function (any five from the list, including acceptable alternatives such as SP, PSW, GPRs).*

Model confidence: high.

Model note: Question asks for FIVE types; the model answer lists five core registers and notes acceptable alternatives. No figure or table is required. The marking guide allocates 10 marks: 5 marks for naming five registers and 5 marks for correct functional descriptions.

QUESTION A9 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (10).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of interfaces to external devices [6 marks]

1. Programmed I/O (polling) interface: The processor directly controls the device through an I/O module and repeatedly checks (polls) the device's status register to see whether it is ready to transfer data. Data are moved between the CPU and the device via the I/O module under direct program control. It is simple and needs no special interrupt hardware, but it wastes CPU time because the processor must busy-wait for slow devices.

2. Interrupt-driven I/O interface: The I/O module signals the processor with an interrupt request when a device is ready or has completed a transfer. The processor suspends its current program, runs an interrupt service routine (ISR) to transfer the data or service the device, then resumes. This frees the CPU from polling, though each transfer still involves CPU intervention and interrupt overhead.

3. Direct Memory Access (DMA) interface: A DMA controller transfers blocks of data directly between the device and main memory without the CPU handling each word. The CPU programs the DMA controller (start address, count, direction) and is interrupted only once, when the whole block transfer is complete. This gives high throughput for fast devices such as disks and network cards.

(Other acceptable interfaces: memory-mapped I/O vs. isolated/port-mapped I/O, serial vs. parallel interfaces, or synchronous vs. asynchronous interfaces, provided each is described.)

(b) Two types of interrupt [4 marks]

1. Maskable interrupt: An interrupt that can be disabled (masked) by the processor, e.g. by clearing an interrupt-enable bit or setting a mask register. It is normally used for ordinary I/O devices whose service can be delayed; the CPU ignores it while masked and services it later when unmasked.

2. Non-maskable interrupt (NMI): An interrupt that cannot be disabled by the normal masking mechanism. It is reserved for high-priority or catastrophic events such as power failure, memory parity error or hardware fault, and is always recognised by the CPU so that urgent action can be taken.

(Alternative acceptable pairing: hardware interrupt vs. software/trap interrupt, or internal (exception) vs. external interrupt, if each is explained.)

Unofficial marking guide:

- *1 mark: naming the first interface type (e.g. programmed I/O/polling)*
- *1 mark: describing how the first interface operates and its CPU involvement/limitation*
- *1 mark: naming the second interface type (e.g. interrupt-driven I/O)*
- *1 mark: describing how the second interface operates (interrupt request, ISR, resume)*
- *1 mark: naming the third interface type (e.g. DMA)*
- *1 mark: describing how the third interface operates (direct memory transfer, single completion interrupt)*
- *1 mark: naming/defining the first interrupt type (e.g. maskable)*
- *1 mark: explaining its behaviour (can be disabled/masked, used for ordinary I/O)*
- *1 mark: naming/defining the second interrupt type (e.g. non-maskable)*
- *1 mark: explaining its behaviour (cannot be masked, used for urgent/fatal events)*

© Hak Cipta Universiti Teknologi MARA

Model confidence: high.
Model note: Question text is complete; no figure or table is required. Alternative valid interface/interrupt pairings are accepted as indicated in the answer, since the paper does not prescribe specific types.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

a) Answer: 156510 = 61D16

- *1 mark for repeated division by 16.*
- *1 mark for correct remainders.*
- *1 mark for reading remainders in reverse order.*
- *1 mark for the correct hexadecimal digits.*
- *1 mark for the base subscript.*

b) Answer: BDC16 = 303610

- *1 mark for place values 16^2, 16^1 and 16^0.*
- *1 mark for correct digit values.*
- *1 mark for correct products.*
- *1 mark for correct addition.*
- *1 mark for the final decimal value.*

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (20).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 1's extracted text reads z=A2-B*C+jl_ D+5, which is garbled. The most conservative reading consistent with the instruction set (which has ADD, SUB, MUL, DIV but no square-root or exponent instruction) is z = (A² − B·C) / (D + 5). The jl_ is taken as a division bar /, A2 as A² (computed as A·A), and D+5 as the denominator. All three programs below compute this expression and leave the result in z. (If the intended equation differs, the same structure applies; see notes.)

Three-address program (each instruction names two sources and one destination; A, B, C, D are fixed registers, x, y, z are temporaries):

```
MUL (A, A, x)     ; x = A * A
MUL (B, C, y)     ; y = B * C
SUB (x, y, x)     ; x = A^2 - B*C   (numerator)
ADD (D, 5, y)     ; y = D + 5       (denominator)
DIV (x, y, z)     ; z = (A^2 - B*C) / (D + 5)
```

The three-address form is the most compact: one instruction per operation, no explicit data movement, because operands and result are all named in the instruction.

One-address program (an implicit accumulator; LOAD/STOR move between memory/registers and the accumulator, ADD p etc. do ACC ← ACC op p):

```
LOAD A          ; ACC = A
MUL A           ; ACC = A * A
STOR x          ; x = A^2
LOAD B          ; ACC = B
MUL C           ; ACC = B * C
STOR y          ; y = B*C
LOAD x          ; ACC = A^2
SUB y           ; ACC = A^2 - B*C
STOR x          ; x = numerator
LOAD D          ; ACC = D
ADD 5           ; ACC = D + 5
STOR y          ; y = denominator
LOAD x          ; ACC = numerator
DIV y           ; ACC = numerator / denominator
STOR z          ; z = final answer
```

The one-address form needs explicit LOAD/STOR because only one operand (the accumulator) is implicit; intermediate results must be spilled to x and y.

Zero-address program (a stack machine; PUSH p pushes operand p, POP p pops the top into p, and ADD/SUB/MUL/DIV pop two operands and push the result):

```
PUSH A          ; stack: A
PUSH A          ; stack: A, A
MUL             ; stack: A*A
PUSH B          ; stack: A*A, B
PUSH C          ; stack: A*A, B, C
MUL             ; stack: A*A, B*C
SUB             ; stack: A*A - B*C
PUSH D          ; stack: num, D
PUSH 5          ; stack: num, D, 5
ADD             ; stack: num, D+5
DIV             ; stack: num / (D+5)
POP z           ; z = final answer
```

The zero-address form uses the stack for all operands and results; note the operand order (push

© Hak Cipta Universiti Teknologi MARA

numerator first, then denominator) so that DIV computes numerator ÷ denominator.

Summary: all three programs compute z = (A² − B·C)/(D + 5) and store the final result in z; they differ only in how operands are addressed (explicitly named, via an accumulator, or via a stack).

Unofficial marking guide:

• 2 marks: correct interpretation/statement of the equation to be computed (A² − B·C)/(D + 5), including A² as A·A and D+5 as denominator

• 6 marks: correct three-address program — 1 mark each for computing A·A, B·C, the subtraction (numerator), D+5 (denominator), the division, and storing the result in z

• 6 marks: correct one-address program — 1 mark each for correct use of LOAD/STOR to form A², B·C, numerator, denominator, division, and final STOR z (accumulator semantics correct)

• 6 marks: correct zero-address program — 1 mark each for correct PUSH sequence for A², correct PUSH/MUL for B·C, correct SUB for numerator, correct PUSH/ADD for D+5, correct DIV with operands in the right order, and POP z for the final result

Model confidence: medium.

*Model note: The equation in Figure 1 is OCR-garbled ('z=A2-B*C+jl_ D+5'). I conservatively read it as z = (A² − B·C)/(D + 5), since the instruction set has no exponent or square-root operation and A² must be computed as A·A. If the actual figure shows a different expression (e.g. a different denominator or an added term), the same three program structures apply but the exact instruction sequences and mark allocation per step would change. The register names A, B, C, D are treated as fixed registers and x, y, z as temporaries, with z holding the final answer, per the question. This is an unofficial model answer, not a marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA